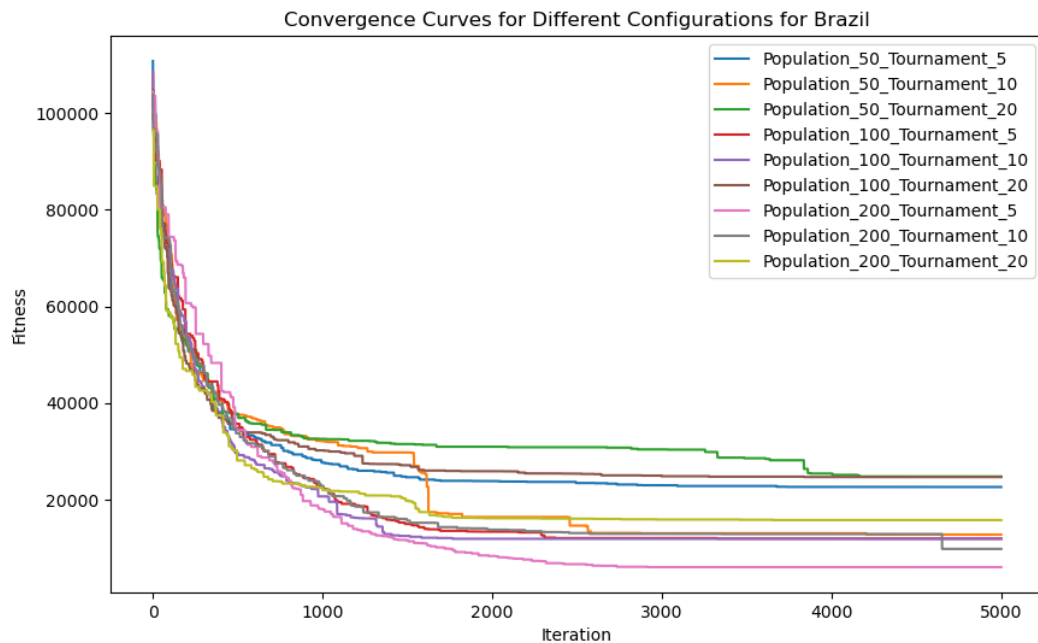
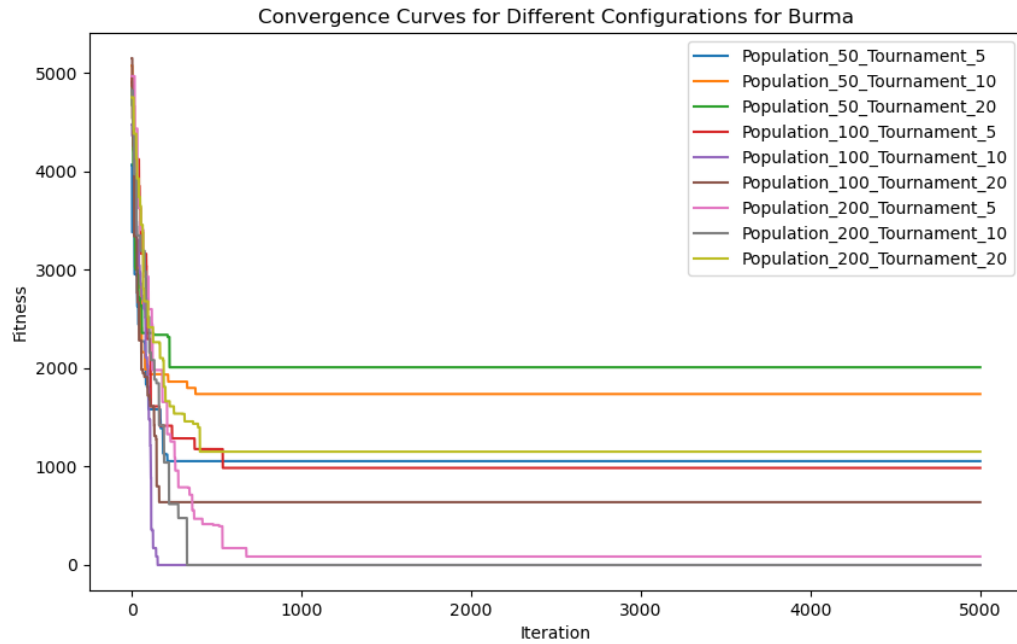


Report

Experiment Execution

Experiments are executed for population size 50, 100 and 200 also for Tournament size with 5, 10 and 20. The analysis from experiments is as population and tournaments gets increase the time for execution is getting increase exponentially. Also, with the increase of population and tournament the best fitness getting increase. So, performance of algorithm is getting affected negatively with it.



Question1: Which combination of parameters produces the best results?

Population 200 and tournament count 5 is generating the best result for this algorithm. It is visible that by increasing the population the performance getting better and cost is getting reduced. Also, by increasing tournament cost is getting increase. So, from experiments this is the best possible combination.

Question 2: What do you think is the reason for your findings in Question 1?

It is visible that by increasing the population the performance getting better and cost is getting reduced. Also, by increasing tournament cost is getting increase. So, as we increase the number of populations the cost will get reduced.

Question 3: How do each of the parameter settings influence the performance of the algorithm?

By increasing the population, the performance getting better and cost is getting reduced. Also, by increasing tournament cost is getting increase. So, as we increase the number of populations the cost will get reduced. Also, by increasing the tournaments the cost is getting increase.

Question 4: Can you think of a local heuristic function to add?

Consider if adding a local heuristic function, such as 2-opt or 3-opt local search, could further improve the quality of solutions obtained by the EA. This involves refining the solutions locally after they are generated.

Question 5: Can you think if any variation for this algorithm to improve your results? Explain your answer.

We can explore potential variations or extensions of the algorithm. This could involve tweaking the existing operators, introducing new operators, or combining the EA with other optimization techniques.

Question 6: Do you think of any any other nature inspired algorithms that might have provided better results?

The other nature-inspired algorithms like Ant Colony Optimization, Simulated Annealing, or Genetic Algorithms could potentially outperform the EA. But, There is no one-size-fits-all answer. The effectiveness of these algorithms depends on the problem characteristics and the effort spent on fine-tuning their parameters. It is common to experiment with multiple algorithms and select the one that performs well for a particular problem or problem instance.

Code

```
import xml.etree.ElementTree as ET
import numpy as np
import numpy as np
import random

def parse_xml(xml_string):
    # parse the whole xml file and generate the distance matrix out of it
    root = ET.fromstring(xml_string)
    vertices = root.findall("./vertex")
    num_vertices = len(vertices)
    distance_matrix = np.zeros((num_vertices, num_vertices))
    for i, vertex in enumerate(vertices):
        edges = vertex.findall("./edge")
        for edge in edges:
            neighbor_index = int(edge.text)
            cost = float(edge.get("cost"))
            distance_matrix[i, neighbor_index] = cost
    return distance_matrix

def initialize_population(size, num_locations):
    # Generate an initial population of random solutions
    population = [np.random.permutation(num_locations) + 1 for _ in range(size)]
    return population

def evaluate_fitness(solution, distance_matrix):
    # Calculate the fitness of a solution based on the provided cost formula
    n = len(solution)
    cost = sum(distance_matrix[solution[i] - 1, solution[(i + 1) % n] - 1] for i in range(n))
    return cost

def tournament_selection(population, tournament_size, distance_matrix):
    # Perform tournament selection to choose a parent
    selected_candidates = random.sample(population, tournament_size)
    fitness_values = [evaluate_fitness(candidate, distance_matrix) for candidate in selected_candidates]
    return selected_candidates[np.argmin(fitness_values)]

def single_point_crossover(parent_a, parent_b):
    # Implement single-point crossover
    crossover_point = random.randint(1, len(parent_a) - 1)
    child_a = np.hstack((parent_a[:crossover_point], parent_b[crossover_point:]))
    child_b = np.hstack((parent_b[:crossover_point], parent_a[crossover_point:]))
    return child_a, child_b

def swap_mutation(solution):
    # Implement swap mutation
    mutation_points = random.sample(range(len(solution)), 2)
    solution[mutation_points[0]], solution[mutation_points[1]] = (
```

```

        solution[mutation_points[1]],
        solution[mutation_points[0]],
    )
    return solution

def replace_worst(population, new_solution, distance_matrix):
    # Replace the worst solution in the population with the new solution
    fitness_values = [evaluate_fitness(candidate, distance_matrix) for candidate in population]
    worst_index = np.argmax(fitness_values)
    population[worst_index] = new_solution

def evolutionary_algorithm(distance_matrix, population_size=100, tournament_size=10, max_evaluations=10000):
    population = initialize_population(population_size, len(distance_matrix))
    evaluations = 0
    while evaluations < max_evaluations:
        parent_a = tournament_selection(population, tournament_size, distance_matrix)
        parent_b = tournament_selection(population, tournament_size, distance_matrix)
        child_a, child_b = single_point_crossover(parent_a, parent_b)
        child_a = swap_mutation(child_a)
        child_b = swap_mutation(child_b)
        replace_worst(population, child_a, distance_matrix)
        replace_worst(population, child_b, distance_matrix)
        evaluations += 2 # Two new solutions are generated in each iteration

    # Retrieve the best solution found
    best_solution = min(population, key=lambda x: evaluate_fitness(x, distance_matrix))
    return best_solution, evaluate_fitness(best_solution, distance_matrix)

if __name__ == '__main__':
    with open("burma14.xml", 'r') as burma_file:
        burma_data = burma_file.read()

    # Parse XML and get the distance matrix
    D_BURMA = parse_xml(burma_data)

    with open("brazil58.xml", 'r') as brazil_file:
        brazil_data = brazil_file.read()

    # Parse XML and get the distance matrix
    D_BRAZIL = parse_xml(brazil_data)

    best_solution, best_fitness = evolutionary_algorithm(D_BURMA, population_size=100, tournament_size=10,
max_evaluations=10000)

    best_solution, best_fitness = evolutionary_algorithm(D_BRAZIL, population_size=100, tournament_size=10,
max_evaluations=10000)

```